COMP 3700A: Contributing to Open Source

Navigating Large Codebases

Charlotte Curtis

September 28, 2026

Housekeeping

- Do we have notetakers for today's topics?
  - Goal is to have the PR for each topic submitted within **1 week**
  - After submission, I will add a reviewer — you may request specific topic
  - If you are chosen as reviewer, you may decline the invitation
- How are things going on part 1 of the project?
  - Presentations next week!

Today's Goals

- Non-code contributions
- Navigating large codebases

Non-code contributions

- At a certain scale, FOSS projects need all kinds of expertise
- Documentation in its various forms
- UI/UX designers
- Translators
- Bug reporters and triagers
- Your examples

Navigating large codebases

- Situation: you've found a **bite-sized issue** and now you want to solve it
- Where do you even start?
- Lots of manual tracing, but also language server (LSP) and `grep`
- Worth knowing a bit of regular expressions

> *Example: a random issue from Matplotlib*
>
> *More complicated: a random issue from Audacity*

Some basic regex

- **Basic characters**
  - A literal character (e.g. `A`, `x`, `4`, `?`)
  - Some of these need an escape character (`\`)
- **Any of some type**:
  - `\d`: any digit
  - `\w`: any "word" character (`Aa-Zz`, `0-9`, `_`)
  - `\W`: any non-word character
  - `\s`: any whitespace
  - `\S`: any non-whitespace

> *Example: write an expression to find `TODO` items*

Multipliers and grouping stuff

- `*`: 0 or more
- `+`: 1 or more
- `?`: 0 or 1
- `{#}`: exactly `#`
- `[ ]`: any of the things in `[]`
- `(?:)`: exact sequence after the `:`

> *Example: write an expression to find Python class declarations, including* `class X:` *and* `class X(Y):`*, but not other mentions of the word `class``*

Boundaries

- `\b` : A word boundary
- `\B` : Not a word boundary
- `^` : start of a line or string*
- `$` : end of a line or string

* unless inside `[]`, then the `^` character means `NOT`

> *Example: write an expression to find `int` declarations in C++ or Java*

What about AI?

- AI does not ingest your entire codebase, it just `grep`s for you
  - For example, my session log with Copilot about Audacity
- I guess it works, kinda... but I used 1% of my monthly AI credits included with GitHub Education and I still know nothing about the code!
- I recommend you try doing your own tracing rather than taking the LLM shortcut (but definitely use LSP features)

Git tools: who and when

- `git blame filename`: line-by-line listing of commits
- `git log -L :funcname:filename`: history of specific function in file
- `git grep`: Like regular grep, but only searches tracked files and can search previous commits

Coming up next

Wednesday: Truth and Reconciliation day, no classes

Thursday: No lab, come to the Tech Transition Event

Also, no homework since your project part 1 is due Friday!